

通行止め・通行困難地点 公開機能 テスト計画書

バージョン: 1.0 最終更新日: 2026年7月19日 対象: 開発担当者・運用担当者(公開機能の動作確認を行う人) 関連: 設計書 [closures-design-202607.md](#) / 運用手順書 [closures-operations-202607.md](#)

1. このテスト計画書について

通行止め・通行困難地点の**公開機能(公開API: Vercel Function + Vercel Blob)**の動作確認手順をまとめたものです。

この機能は従来の「GitHub で確認して Vercel にデプロイ」という流れでは検証できません。公開APIの実体が Vercel にしかないため、環境ごとにできること・できないことを踏まえ、開発用PCと運用用PCで役割を分けてテストします。

1.1 前提

- アプリは一般公開前のため、既存ユーザーへの影響は考慮不要。本番の Vercel Blob を直接テストに使う(Preview 用の別ストア分離は不要)。
- 全テストは本番環境(<https://minoh-hiking.vercel.app>)を対象に実施できる。

1.2 2台のPCの役割分担

PC	役割	検証すること
開発用PC	開発担当者の視点	API・エラー処理・キャッシュなど仕組みが正しいかを curl や DevTools で確認
運用用PC	運用担当者の視点	運用手順書だけを見て公開作業が完結するかの受け入れテスト。開発ツールは使わない

1.3 環境ごとにできること・できないこと

環境	表示(GET)	公開(POST)	注意
ローカル(python http.server)	X(404→表示なしにフォールバック)	X(E05になる)	UI・反映(localStorage)までは確認可
GitHub Pages	○(本番APIを参照)	○だが本番公開そのもの	github.io からは本番 Vercel API を直接叩く。公開テストは運用用PC側に集約する
Vercel 本番	○	○	テストの主戦場(一般公開前のため直接使用可)

2. 事前準備: テストデータ(開発用PCで作成)

ファイル	内容	用途
test-normal.geojson	通行止め(closed)1点 + 通行困難(difficult)1点、箕面エリア内	正常系
test-updated.geojson	上記から1点を削除したもの	解除(一部削除)の確認
test-empty.geojson	0件の FeatureCollection	全削除警告の確認
test-bad-coords.geojson	経度135.9など範囲外の点を含む	E03(範囲外)の確認
test-dup-id.geojson	同じ id が2点ある	E03(id重複)の確認
test-broken.json	FeatureCollection でない JSON	読み込み時の形式エラー確認

座標の妥当範囲は経度 135.2～135.8/緯度 34.6～35.1(API側で検証)。スキーマの詳細は[設計書 §4.2](#)を参照。

3. 開発用PCのテスト項目(仕組みの検証)

D1. Vercel 設定の確認(最初に一度)

Vercel ダッシュボードで以下を確認する。

- ☐ Blob ストアがプロジェクトに Connect 済み
- ☐ 環境変数 `BLOB_READ_WRITE_TOKEN` が設定されている(Blob 接続で自動設定)
- ☐ 環境変数 `CLOSURES_PUBLISH_TOKEN` が Production に設定されている
- ☐ 最新デプロイが Ready(環境変数の追加・変更後は再デプロイ済みであること)

D2. API 単体テスト(curl)

PowerShell では `curl.exe` を使う(`curl` は `Invoke-WebRequest` の別名のため)。

#	手順	期待結果
D2-1	<code>curl.exe https://minoh-hiking.vercel.app/api/closures</code>	200 + geojson(未公開なら空の FeatureCollection)
D2-2	トークンなしで POST	401(未設定なら 503 → D1へ戻る)
D2-3	誤トークン(<code>x-publish-token: wrong</code>)で POST	401
D2-4	正トークン + test-bad-coords	400 + 理由「...範囲外です」
D2-5	正トークン + test-dup-id	400 + 理由「id が重複...」

#	手順	期待結果
D2-6	正トークン + version なしデータ	400 + 理由「version がありません」
D2-7	正トークン + test-normal(version付き)	200 → 直後の GET で同じ version が返る

コマンド例:

```
# D2-1: GET
curl.exe https://minoh-hiking.vercel.app/api/closures

# D2-2: トークンなし POST
curl.exe -X POST https://minoh-hiking.vercel.app/api/closures -H "Content-Type: application/json" -d "{}"

# D2-3: 誤トークン POST
curl.exe -X POST https://minoh-hiking.vercel.app/api/closures -H "Content-Type: application/json" -H "x-publish-token: wrong" -d "{}"

# D2-7: 正常データの公開(ファイル送信)
curl.exe -X POST https://minoh-hiking.vercel.app/api/closures -H "Content-Type: application/json" -H "x-publish-token: <トークン>" --data-binary "@test-normal.geojson"
```

D3. アプリのローカルUI検証(APIなしの挙動)

ローカル配信(`python -m http.server`)で実施する。

#	手順	期待結果
D3-1	?closure=true なしで起動	「通行止め・通行困難地点」ボタンが 出ない
D3-2	?closure=true 付きで起動 → リロード	ボタン表示、リロード後も維持(sessionStorage)
D3-3	test-broken.json を読み込み	形式エラーのトースト、地図は変化なし
D3-4	test-normal を読み込み	プレビュー表示(赤✕・橙三角、件数トースト)
D3-5	バージョン未変更のまま	「マップに反映」が押せない
D3-6	バージョン変更 → 反映	localStorage に保存、リロード後も表示される

#	手順	期待結果
D3-7	ローカルで「公開」	E05 が出るのが正常 (ローカルにAPIなし)、バックアップ保存の提案が出る
D3-8	読み込み後キャンセル	プレビューが消え反映済み表示に戻る

D4. キャッシュ・オフライン動作(本番URLで)

#	手順	期待結果
D4-1	公開直後にアプリを開き直す	リロード待ちなしで新 version が表示 (no-cache 動作)
D4-2	一度表示後、DevTools でオフラインにして開き直す	SW の closures-cache から最終取得分が表示される
D4-3	公開後、localStorage の minoh-hiking.closure-data が消えている	自己修復(version 一致時の削除)が機能

D5. GitHub Pages 版の確認(GETのみ)

- ☐ GitHub Pages 版アプリを開き、通行止めマーカが表示されること (= 本番APIへのクロスオリジン GET + CORS が機能)
- 公開操作はここからでも本番に飛ぶため、公開テストは運用用PC側に集約する。**

4. 運用用PCのテスト項目(運用手順の受け入れテスト)

原則として**[運用手順書](#)だけを見て**実施する。手順書の記述で迷った箇所は、それ自体を指摘事項として記録する(手順書の検証を兼ねる)。テストデータは開発用PCから受け渡す。

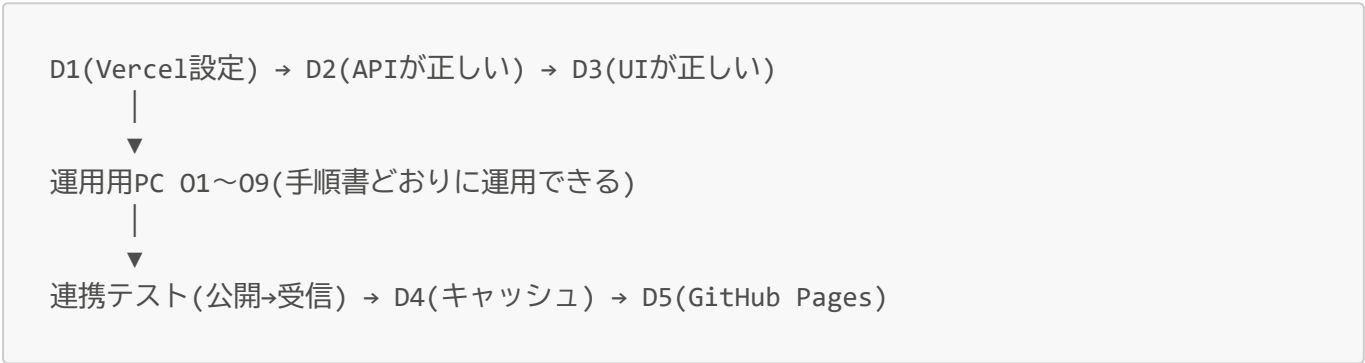
#	手順	期待結果
O1	MapGPS からのリンク(?closure=true)で起動	ホームにボタンが表示される
O2	test-normal を読み込み → 地図で位置・種別・件数を確認	手順書 5章どおりに進められる
O3	バージョンを 日付.連番 形式で変更 → マップに反映	確認ダイアログ → この端末に反映
O4	「公開」 → わざと誤トークン を入力	E01 表示 → 再度「公開」で再入力を求められる(回復手順の確認)
O5	正しいトークンで「公開」	成功メッセージ、パネルが閉じる
O6	アプリを閉じて開き直し、再度公開操作	トークン再入力なし で公開できる(端末保存の確認)
O7	test-updated(1点削除)を新バージョンで公開	削除した地点が消える(全置換の理解確認)

#	手順	期待結果
O8	test-empty(0件)を公開	「全地点が消えます」警告が出る → OK で全消去
O9	最後に test-normal を新バージョンで再公開	表示が復旧(テスト後の原状回復を兼ねる)

5. 2台連携テスト(公開→受信の一气通貫)

- 1. **運用用PC**: 新バージョンで公開(O5)
- 2. **開発用PC**: ユーザー役としてアプリを開き直す → 新しい内容・件数が表示される
- 3. 開発用PCでは何も操作していないこと(受信は開き直しだけで完了)を確認する

6. 実施順序の推奨



先に開発用PCで仕組みを固めてから運用テストに入ると、運用側で問題が出たときに「手順書の問題」か「実装の問題」かを切り分けやすい。

7. 変更履歴

日付	バージョン	内容
2026-07-19	1.0	初版(開発用PC/運用用PCの2台構成によるテスト計画)